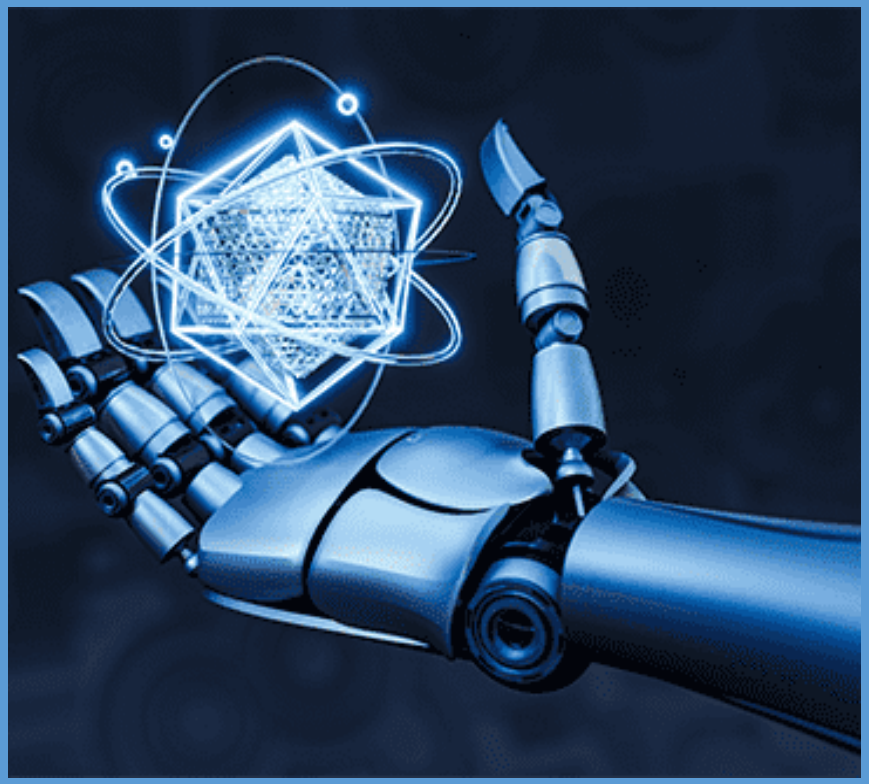
Applied AI & Machine Learning
CS-333

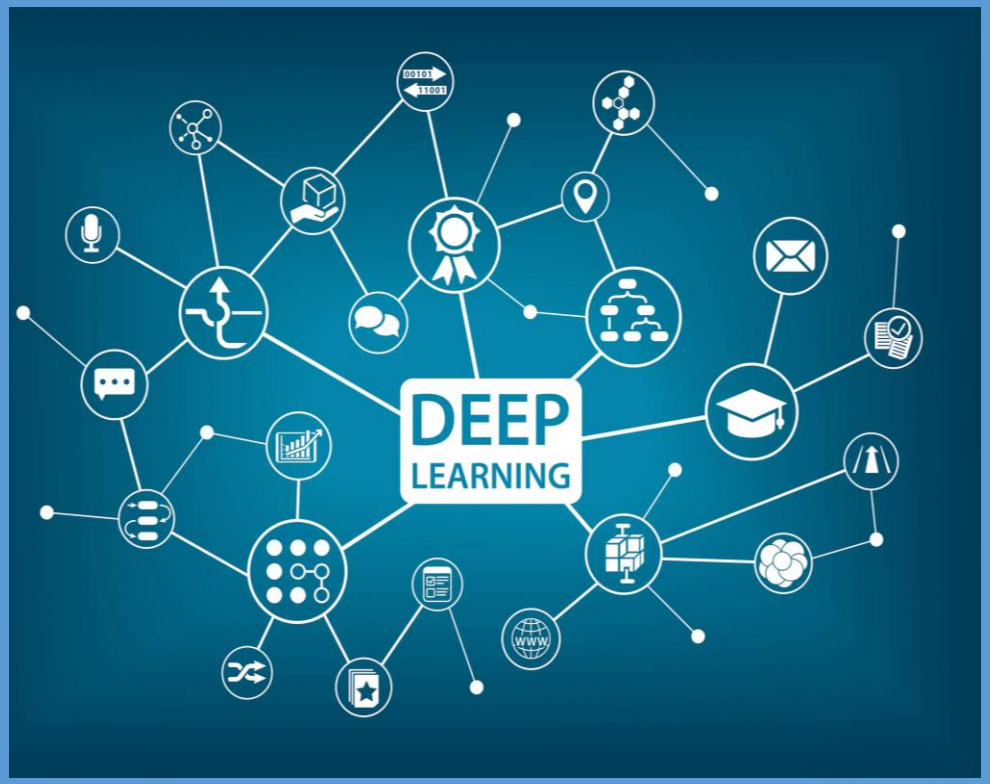
Dr. Abbas Hussain

PNEC, NUST

Lecture 11

Spring 2026





Deep Sequence Modeling

Topics

1. Introduction to Sequential Data (Sequences in the Wild)
2. Recurrent Neural Networks (RNNs)
3. Challenges in RNNs
4. Long Short-Term Memory (LSTM) Networks
5. Limitations of RNNs and LSTMs
6. Introduction to Attention Mechanism

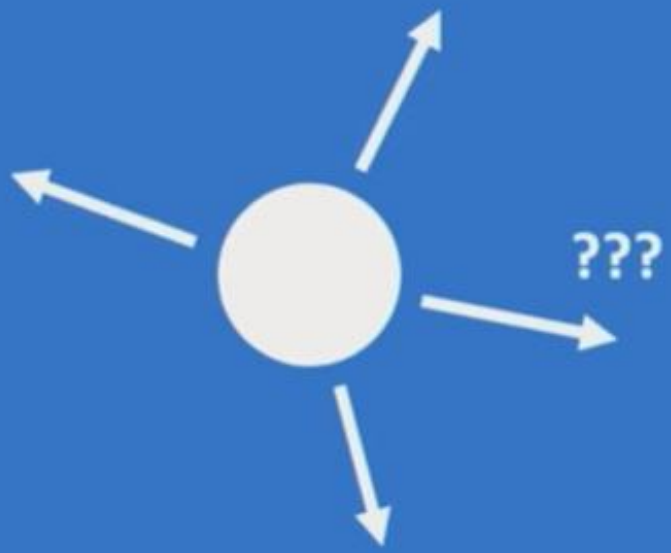
Lecture slides are taken from MIT 6.S191 Intro to deep learning course

Learning Outcomes

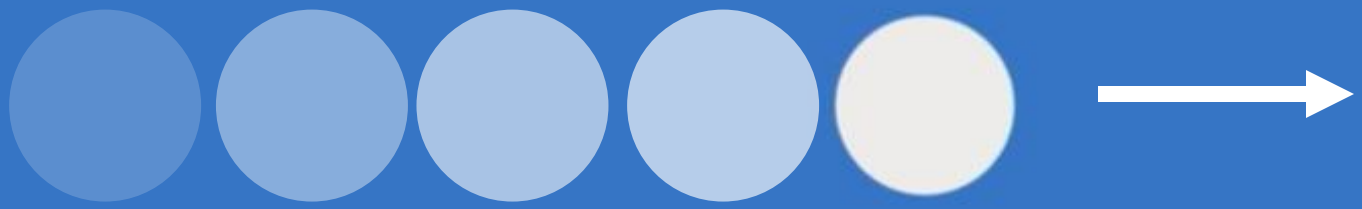
By the end of this lecture, students will be able to:

- Understand the concept of sequential data and its real-world applications (e.g., text, time series)
- Explain the idea of recurrence and how past information influences future predictions
- Describe the architecture and working principle of Recurrent Neural Networks (RNNs)
- Interpret the role of hidden states as memory in sequence modeling
- Identify key challenges in training RNNs, including vanishing and exploding gradients
- Explain how LSTM networks address long-term dependency problems
- Compare RNNs and LSTMs in terms of performance and limitations
- Understand the motivation behind attention mechanisms
- Explain the basic idea of Transformer models and their advantages over RNN-based approaches

Given an image of a ball,
can you predict where it will go next?



Given an image of a ball,
can you predict where it will go next?



Sequences in the Wild

Audio

Sequences in the Wild

characters C S 3 3 3

words APPLIED AI AND MACHINE LEARNING

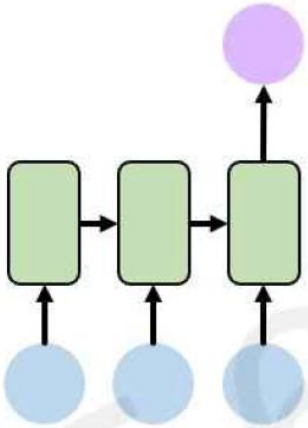
TEXT

Sequences in the Wild

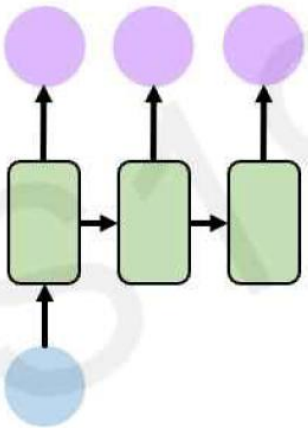
Sequence Modeling Applications

One to One
Binary Classification

"Will I pass this class?"
Student → Pass?

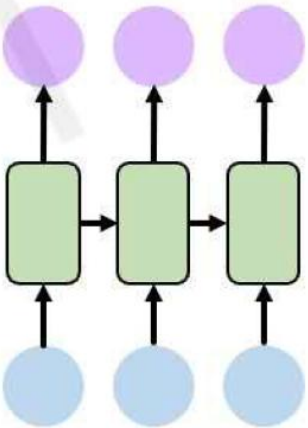


Many to One
Sentiment Classification



One to Many
Image Captioning

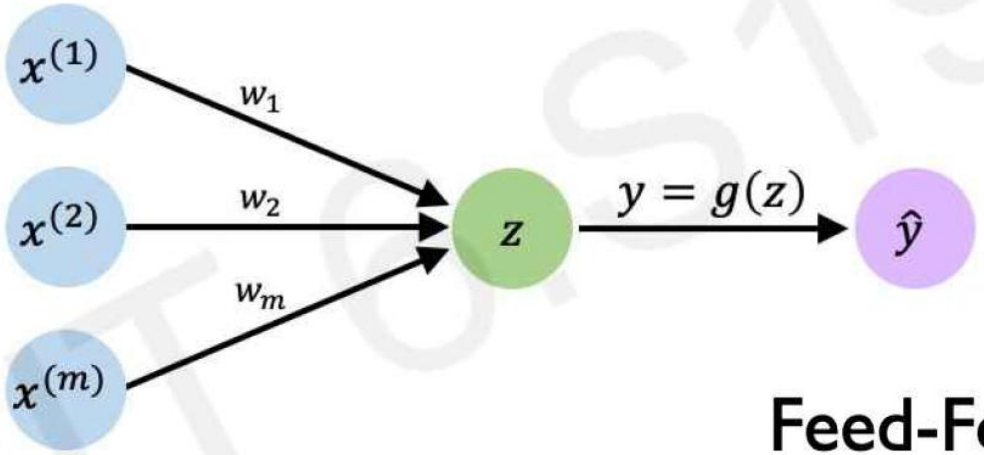
"A baseball player throws a ball."



Many to Many
Machine Translation

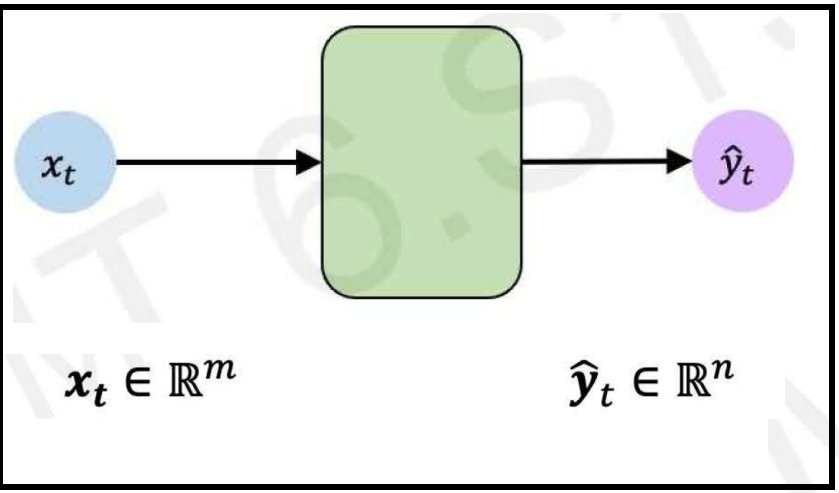
Neurons with Recurrence

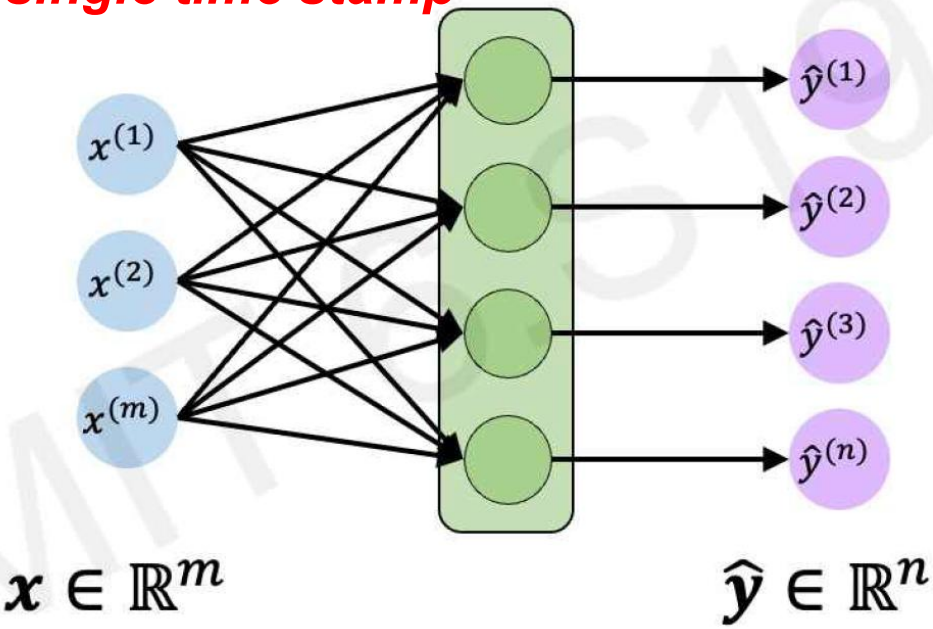
The Perceptron Revisited



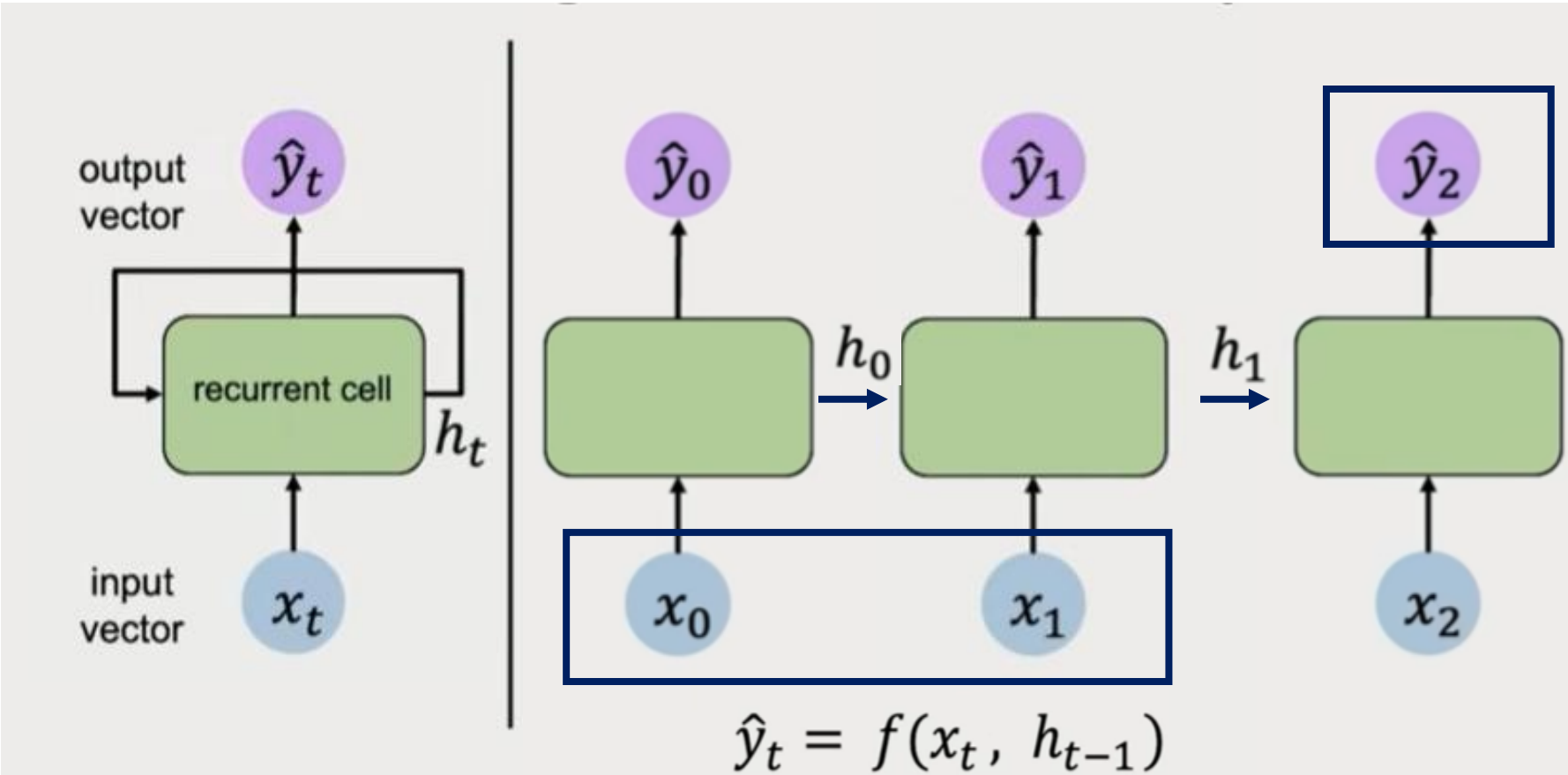
Feed-Forward Networks Revisited

Still operating on static ! means single time stamp





Handling Individual Time Steps

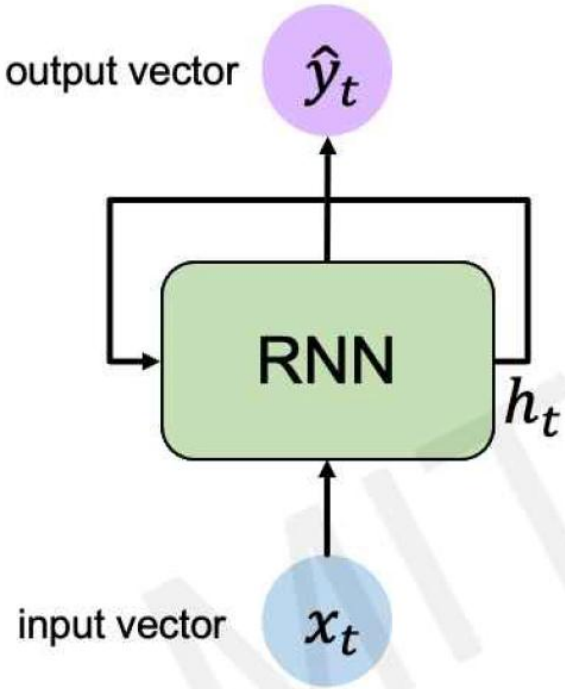


$$\hat{y}_t = f(x_t, h_{t-1})$$

output *input* *Past memory*

Recurrent Neural Networks (RNNs)

Recurrent Neural Networks (RNNs)



Apply a **recurrence relation** at every time step to process a sequence:

$$\boxed{h_t} = \boxed{f_W}(\boxed{x_t}, \boxed{h_{t-1}})$$

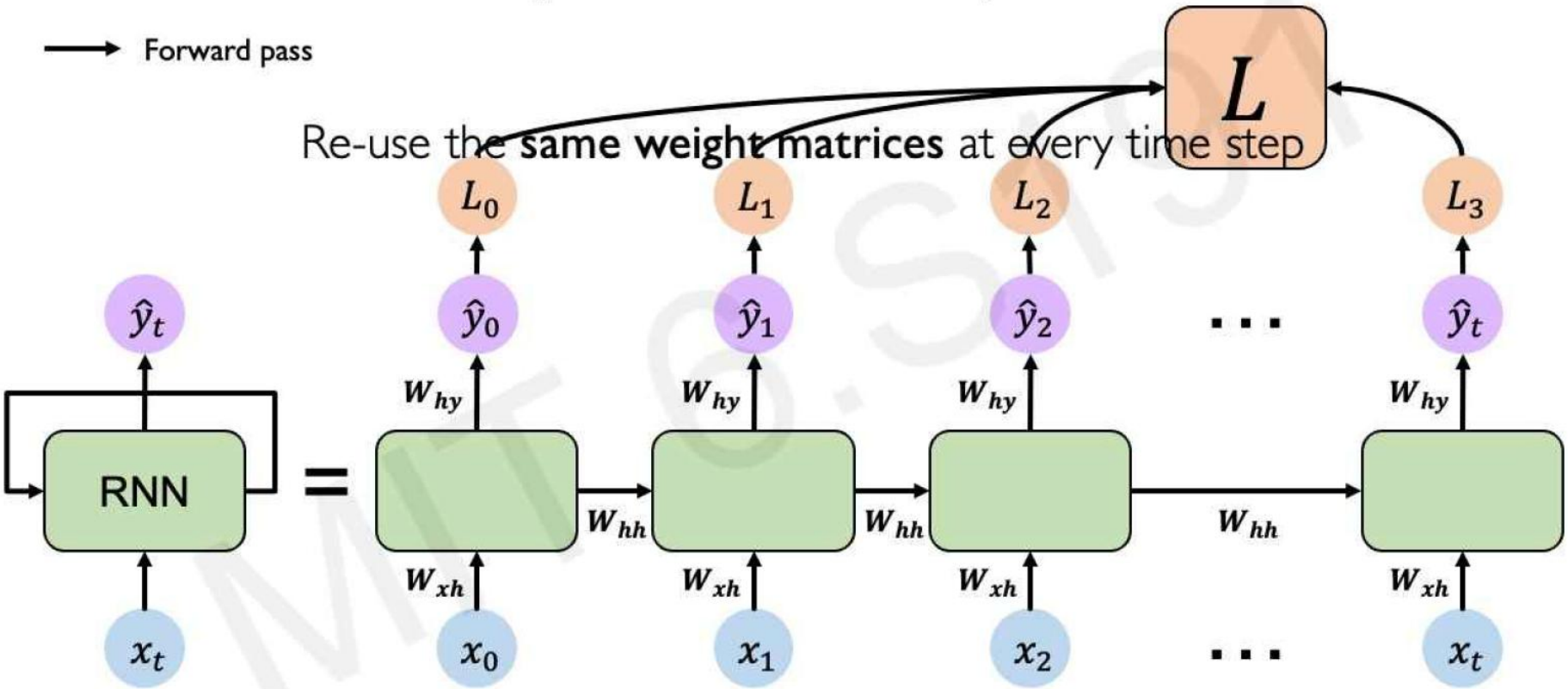
cell state — function with weights W — input — old state

Note: the same function and set of parameters are used at every time step

RNNs have a **state**, h_t, that is updated **at each time step** as a sequence is processed

MIT 6.S191

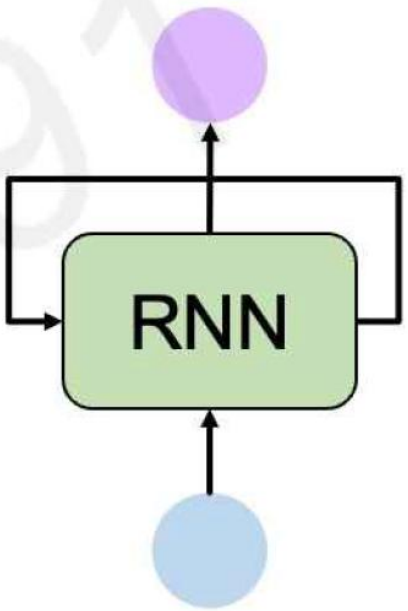
RNNs: Computational Graph Across Time



Sequence Modeling: Design Criteria

To model sequences, we need to:

1. Handle **variable-length** sequences
2. Track **long-term** dependencies
3. Maintain information about **order**
4. **Share parameters** across the sequence



Recurrent Neural Networks (RNNs) meet these sequence modeling design criteria

MIT 6.S191

key challenges faced by RNNs

1. Vanishing Gradient Problem

Gradients can become extremely small as they are propagated backward through time, making it difficult for the network to learn from distant past information. This hinders the RNN's ability to capture long-term dependencies.

2. Exploding Gradient Problem

Gradients can become extremely large, leading to unstable training and convergence issues. This can result in a loss function that increases rapidly and prevents the network from learning effectively.

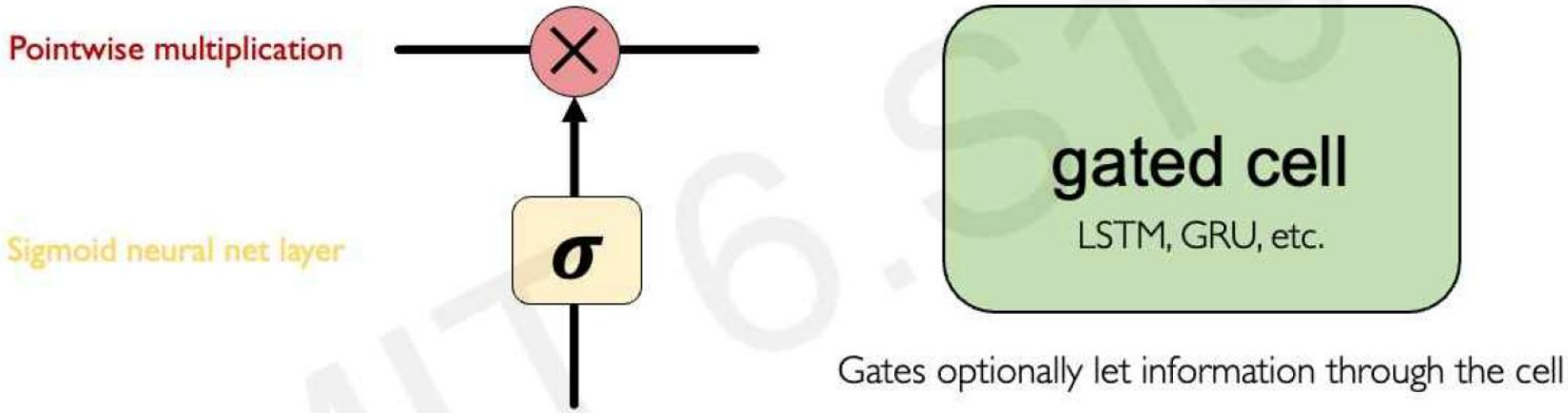
Gradient Clipping

3. Lack of Parallelism

The inherently sequential nature of RNNs limits their parallel processing capabilities, making them less efficient for deployment on certain hardware architectures.

Gating Mechanisms in Neurons

Idea: use **gates** to selectively **add** or **remove** information within **each recurrent unit with**



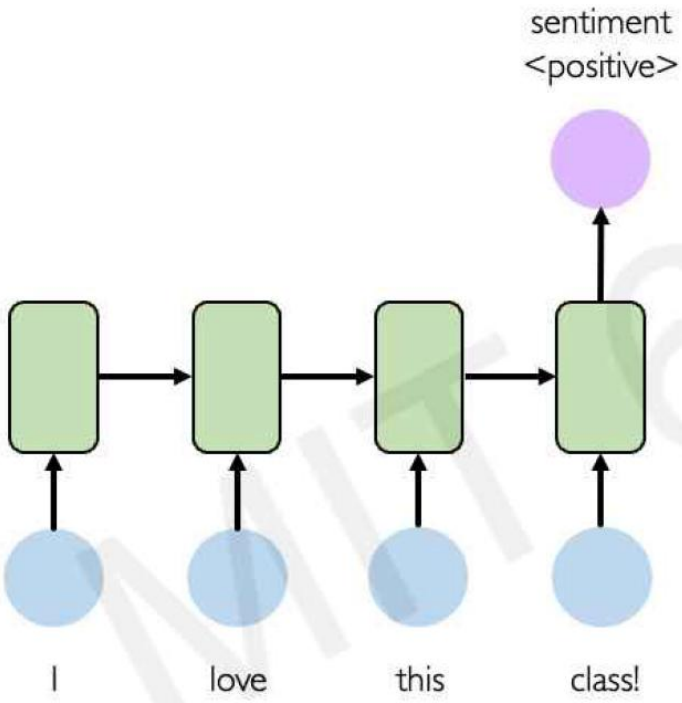
Gates optionally let information through the cell

Long Short Term Memory (LSTMs) networks rely on a gated cell to track information throughout many time steps.

Challenges faced by LSTM

1. Complex and slower to train and demands more memory.
2. Still not suitable for very long-range dependencies.
3. Limits the ability to process data in parallel which slows down training.

Limitations of Recurrent Models



Limitations of RNNs

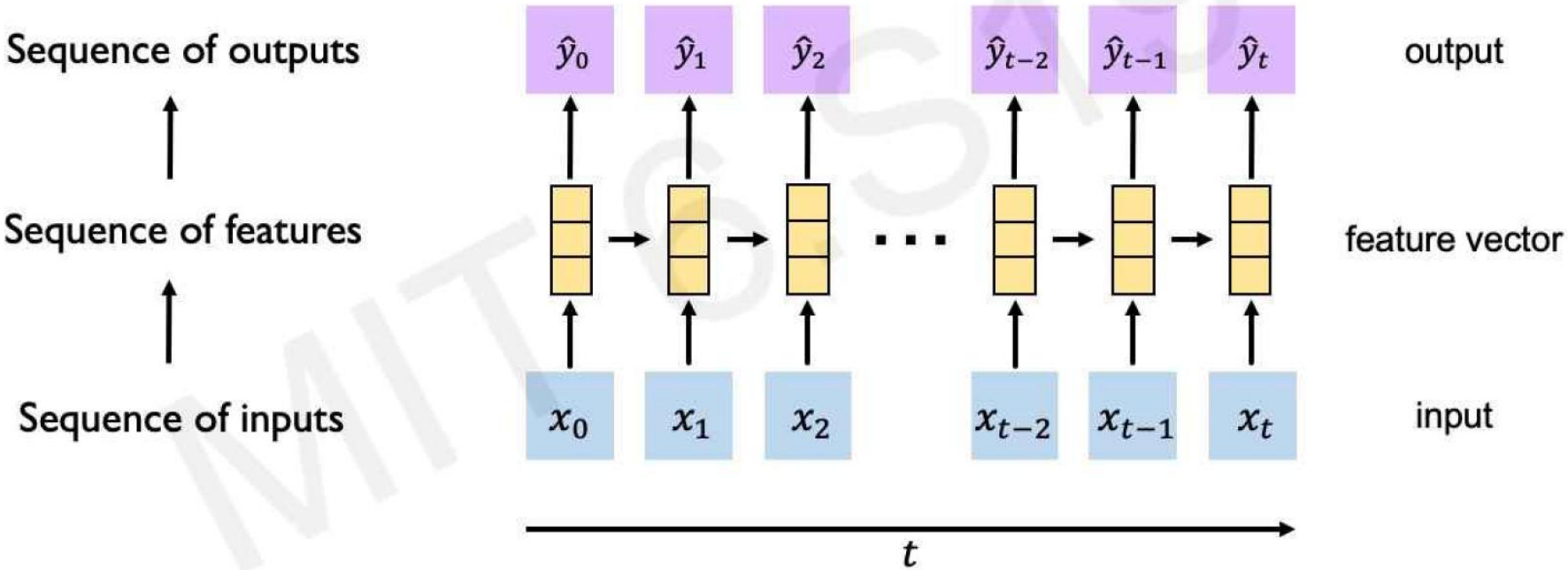
- Encoding bottleneck
- Slow, no parallelization
- Not long memory

MIT 6.S191

Attention Is All You Need

Goal of Sequence Modeling

RNNs: recurrence to model sequence dependencies



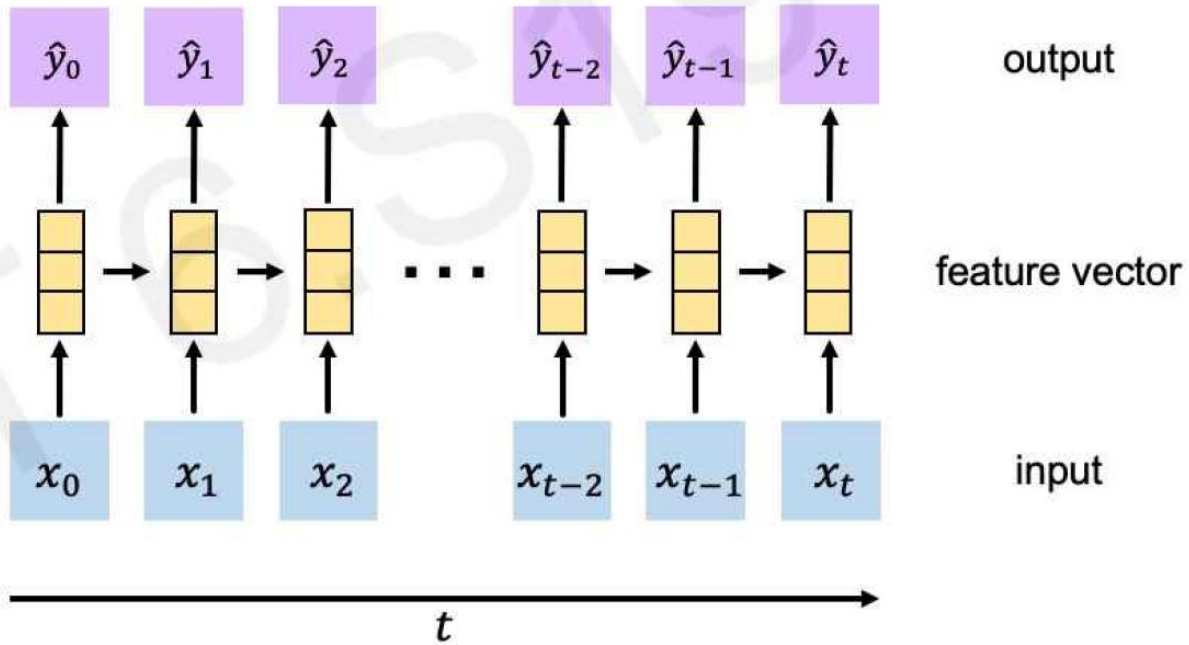
MITS491

Goal of Sequence Modeling

Can we eliminate the need for recurrence entirely?

Desired Capabilities

- Continuous stream
- Parallelization
- Long memory



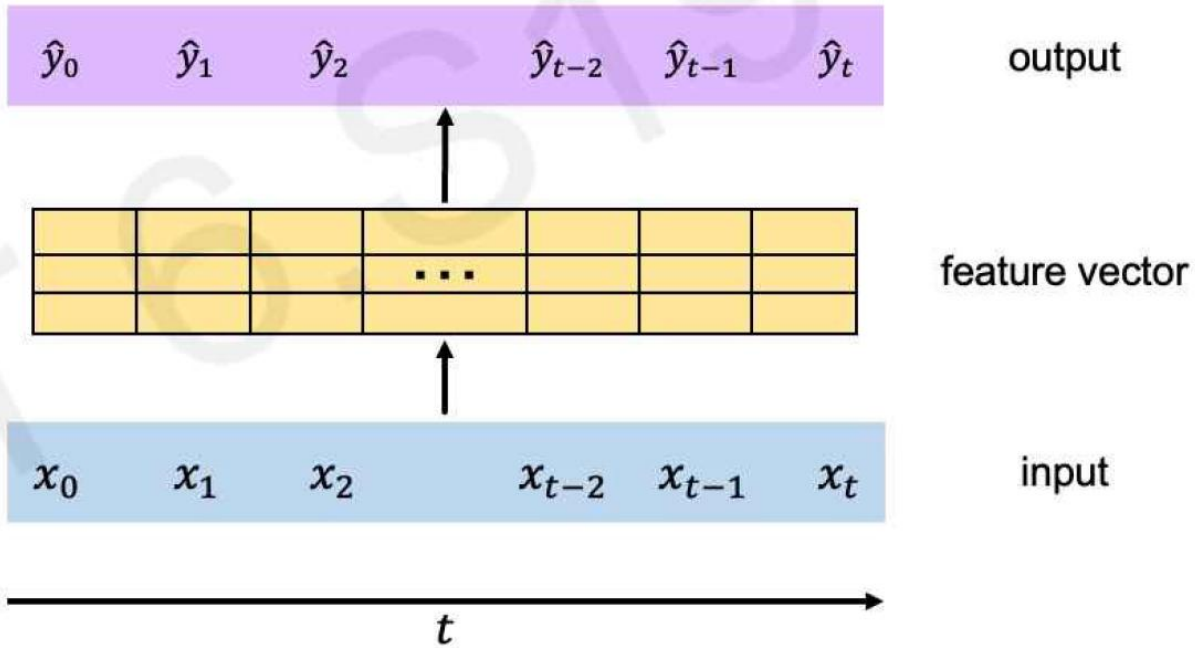
MIT6.S191

Goal of Sequence Modeling

Can we eliminate the need for recurrence entirely?

Desired Capabilities

- Continuous stream
- Parallelization
- x_0 Long memory



MIT 6.S191

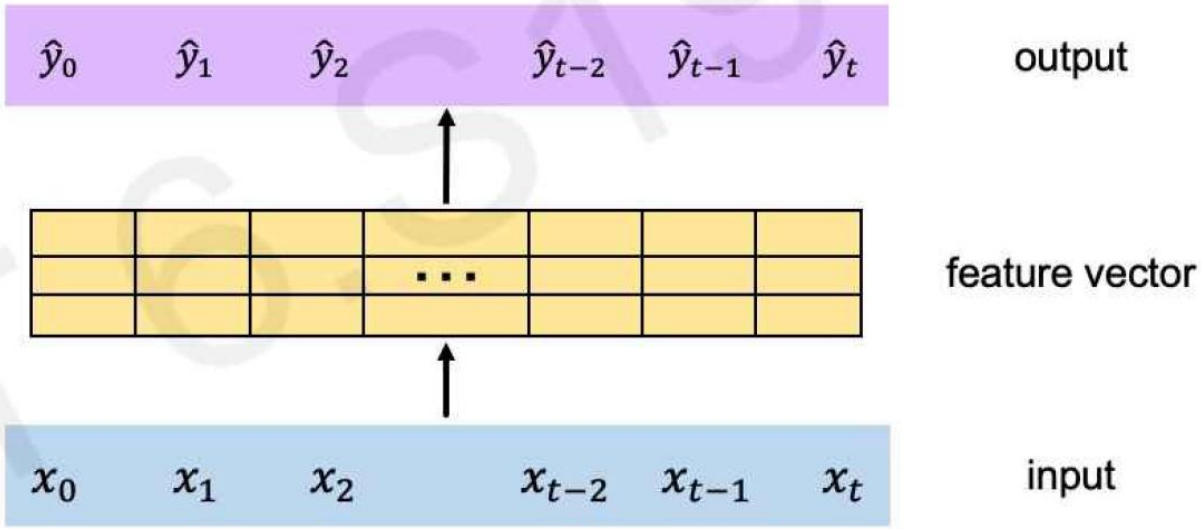
Goal of Sequence Modeling

Idea 1: Feed everything into dense network

- ✓ No recurrence
- ✗ Not scalable
- ✗ No order
- ✗ No long memory

Idea: Identify and attend to what's important

Can we eliminate the need for recurrence entirely?



MIT 6.S191

Intuition Behind Self-Attention

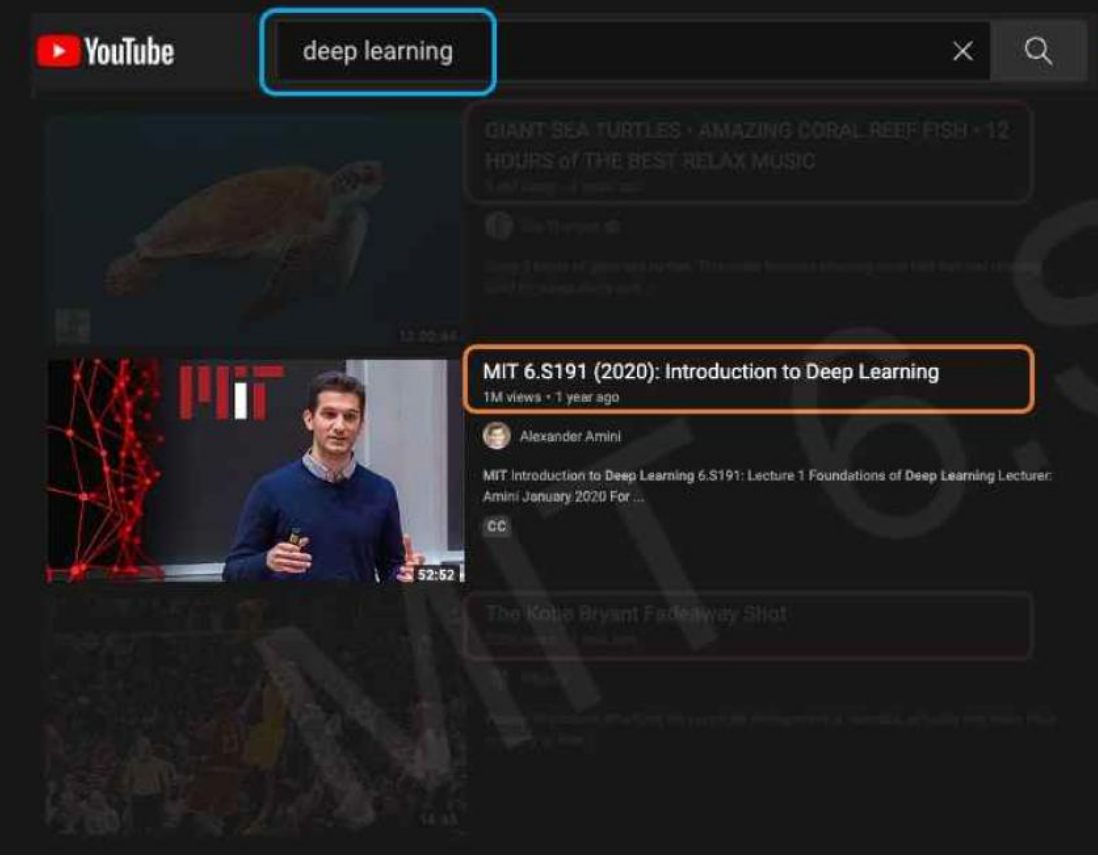
Attending to the most important parts of an input.

Intuition Behind Self-Attention

Attending to the most important parts of an input.

1. Identify which parts to attend to
2. Extract the features with high attention

Understanding Attention with Search



Query (Q)

Key (K_1)

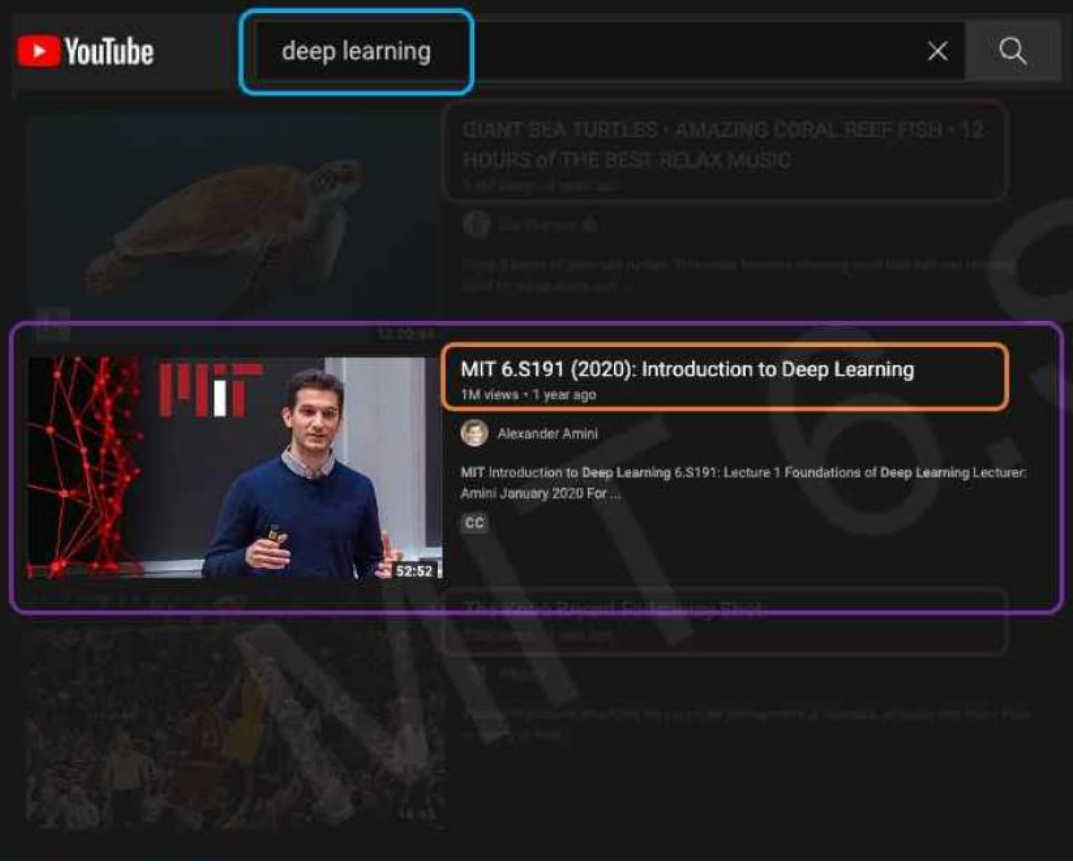
Key (K_2)

Key (K_3)

How similar is the key to the query?

1. **Compute attention mask:** how similar is each key to the desired query?

Understanding Attention with Search



Query (Q)

Key (K_1)

Key (K_2)

Value (V)

Key (K_3)

2. **Extract values based on attention:**
Return the values highest attention

Learning Self-Attention with Neural Networks

Goal: identify and attend to most important features in input.

1. Encode **position** information
2. Extract **query, key, value** for search
3. Compute **attention weighting**
4. Extract **features with high attention**

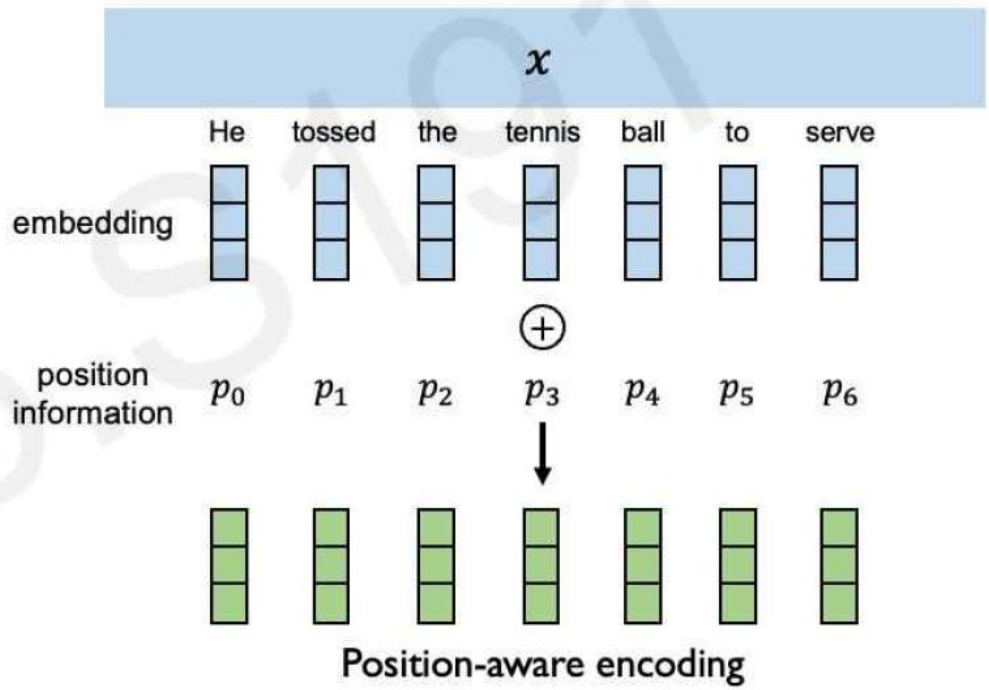
Data is fed in all at once! Need to encode position information to understand order.

MIT 6.S191

Learning Self-Attention with Neural Networks

Goal: identify and attend to most important features in input.

1. Encode **position** information
2. Extract query, key, value for search
3. Compute attention weighting
4. Extract features with high attention



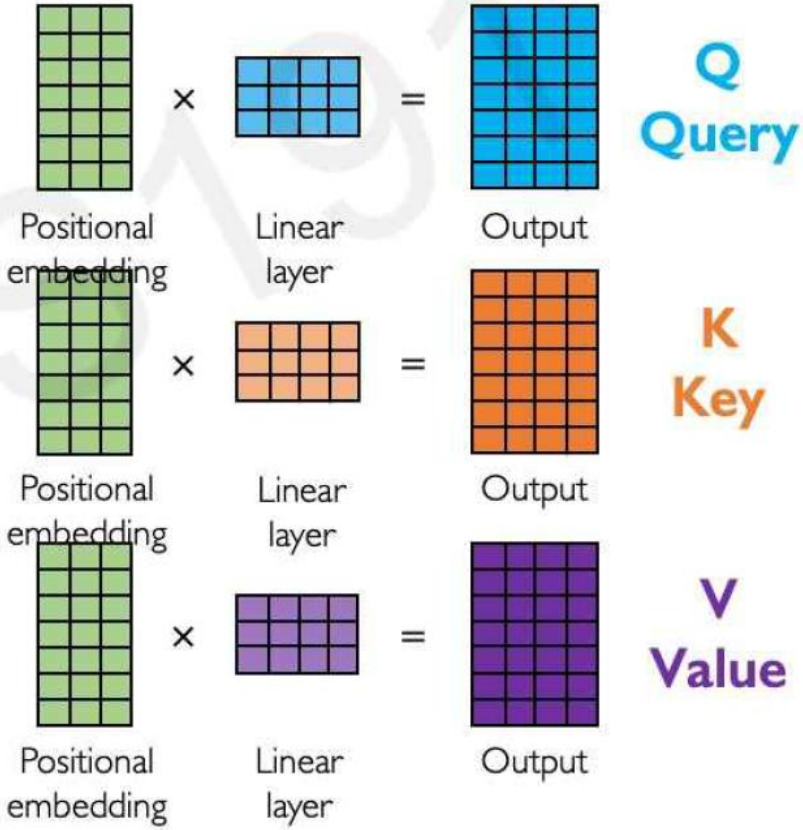
Data is fed in all at once! Need to encode position information to understand order.

MIT 6.S191

Learning Self-Attention with Neural Networks

Goal: identify and attend to most important features in input.

1. Encode **position** information
2. Extract **query, key, value** for search
3. Compute attention weighting
4. Extract features with high attention



MIT 6.S191

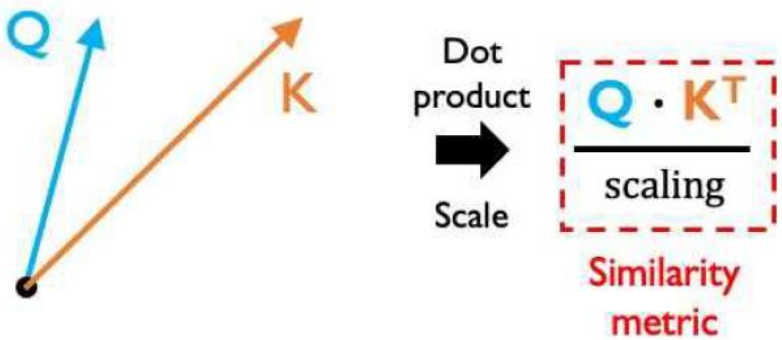
Learning Self-Attention with Neural Networks

Goal: identify and attend to most important features in input.

1. Encode **position** information
2. Extract **query, key, value** for search
3. Compute **attention weighting**
4. Extract features with high attention

Attention score: compute pairwise similarity between each query and key

How to compute similarity between two sets of features?



Also known as the "cosine similarity"

MIT 6.S191

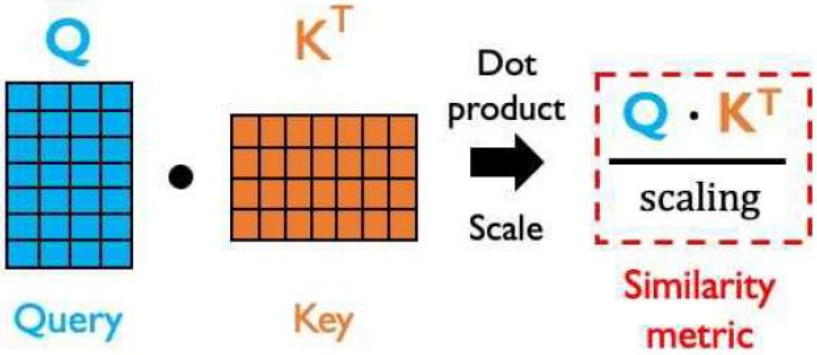
Learning Self-Attention with Neural Networks

Goal: identify and attend to most important features in input.

1. Encode **position** information
2. Extract **query, key, value** for search
3. Compute **attention weighting**
4. Extract features with high attention

Attention score: compute pairwise similarity between each query and key

How to compute similarity between two sets of features?



Also known as the "cosine similarity"

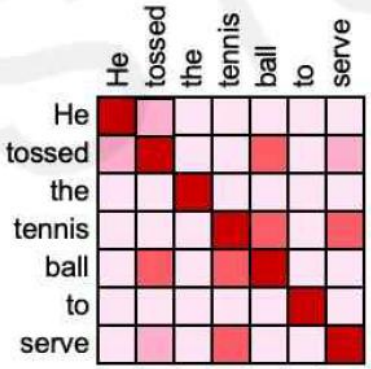
MIT 6.S191

Learning Self-Attention with Neural Networks

Goal: identify and attend to most important features in input.

1. Encode **position** information
2. Extract **query, key, value** for search
3. Compute **attention weighting**
4. Extract features with high attention

Attention weighting: where to attend to! How similar is the key to the query?



$$softmax\left(\frac{Q \cdot K^T}{scaling}\right)$$

Attention weighting

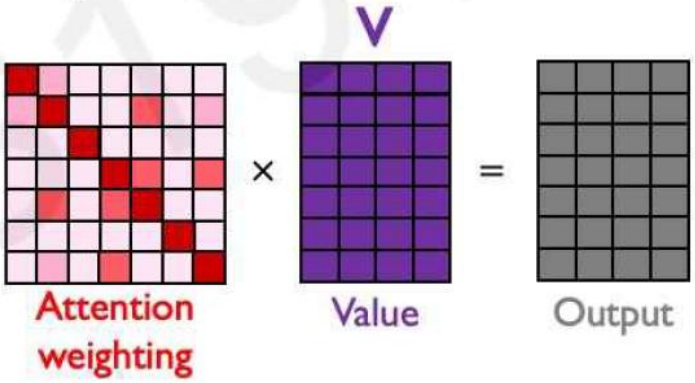
MIT 6.S191

Learning Self-Attention with Neural Networks

Goal: identify and attend to most important features in input.

1. Encode **position** information
2. Extract **query, key, value** for search
3. Compute **attention weighting**
4. Extract **features with high attention**

Last step: self-attend to extract features



$$softmax\left(\frac{Q \cdot K^T}{scaling}\right) \cdot V = A(Q, K, V)$$

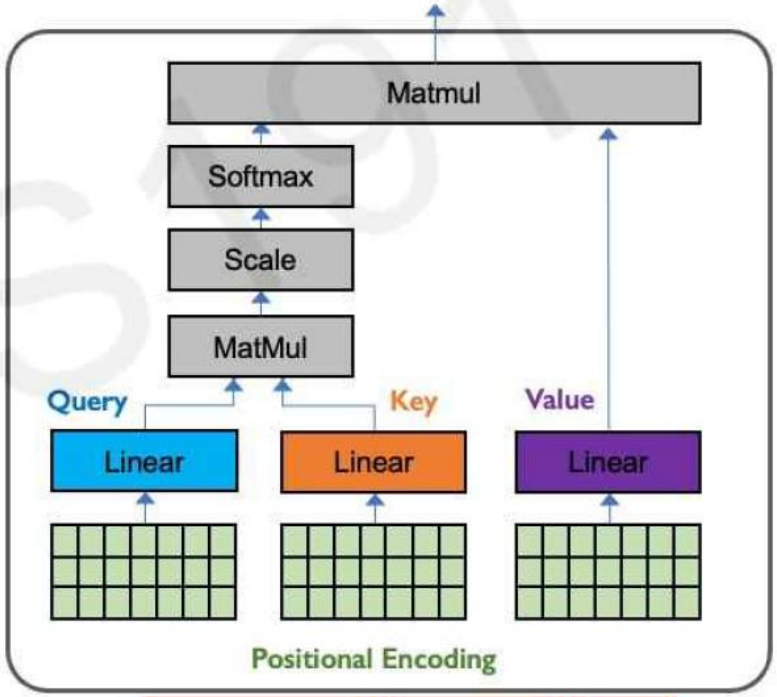
MIT 6.S191

Learning Self-Attention with Neural Networks

Goal: identify and attend to most important features in input.

1. Encode **position** information
2. Extract **query, key, value** for search
3. Compute **attention weighting**
4. Extract **features with high attention**

These operations form a self-attention head that can plug into a larger network.
Each head attends to a different part of input.

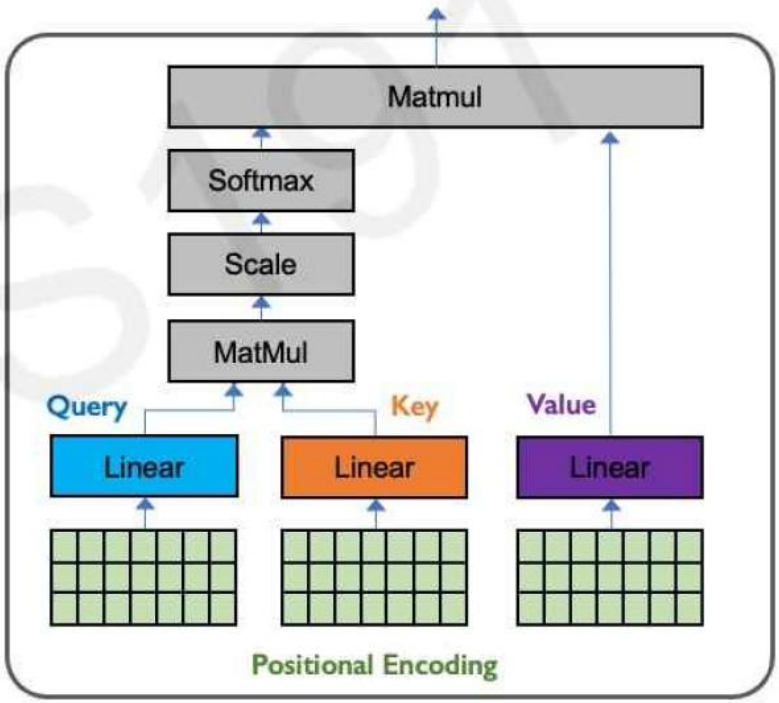


$$softmax\left(\frac{Q \cdot K^T}{scaling}\right) \cdot V$$

Learning Self-Attention with Neural Networks

Goal: identify and attend to most important features in input.

1. Encode **position** information
2. Extract **query, key, value** for search
3. Compute **attention weighting**
4. Extract **features with high attention**

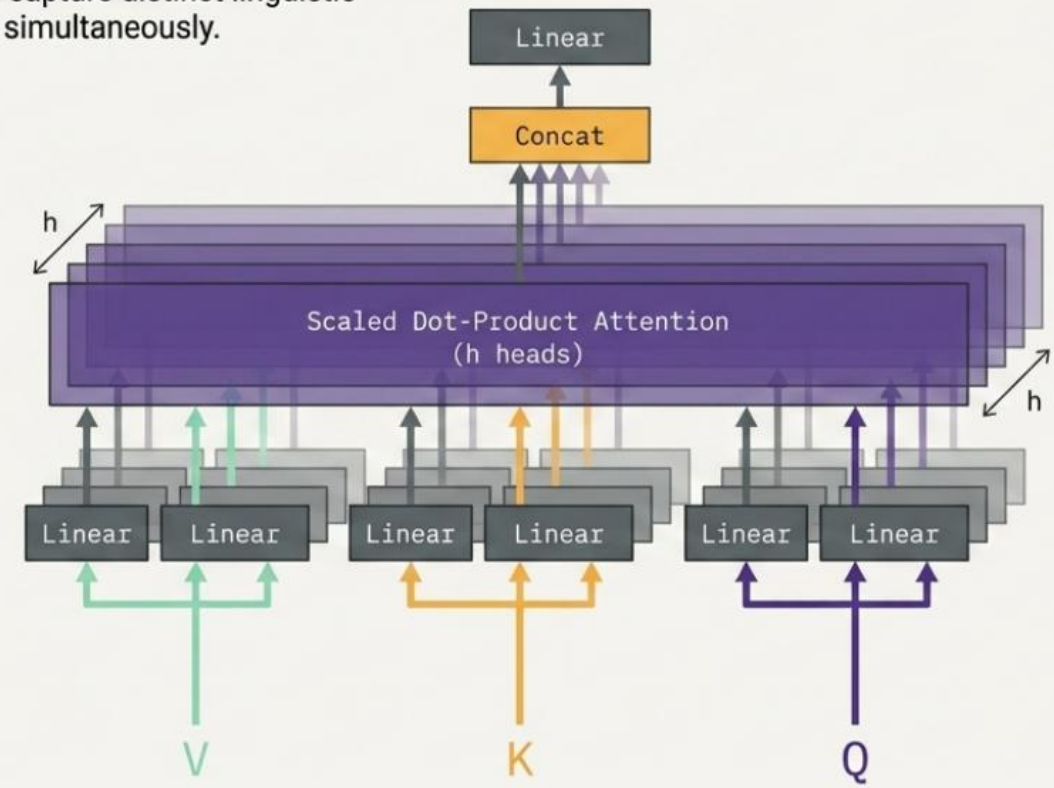


Attention is the foundational building block of the **Transformer** architecture.

MIT 6.S191

Multi-Head Attention Parallelizes Perspectives

A single attention function captures only one notion of similarity. We run h separate heads in parallel to capture distinct linguistic phenomena simultaneously.



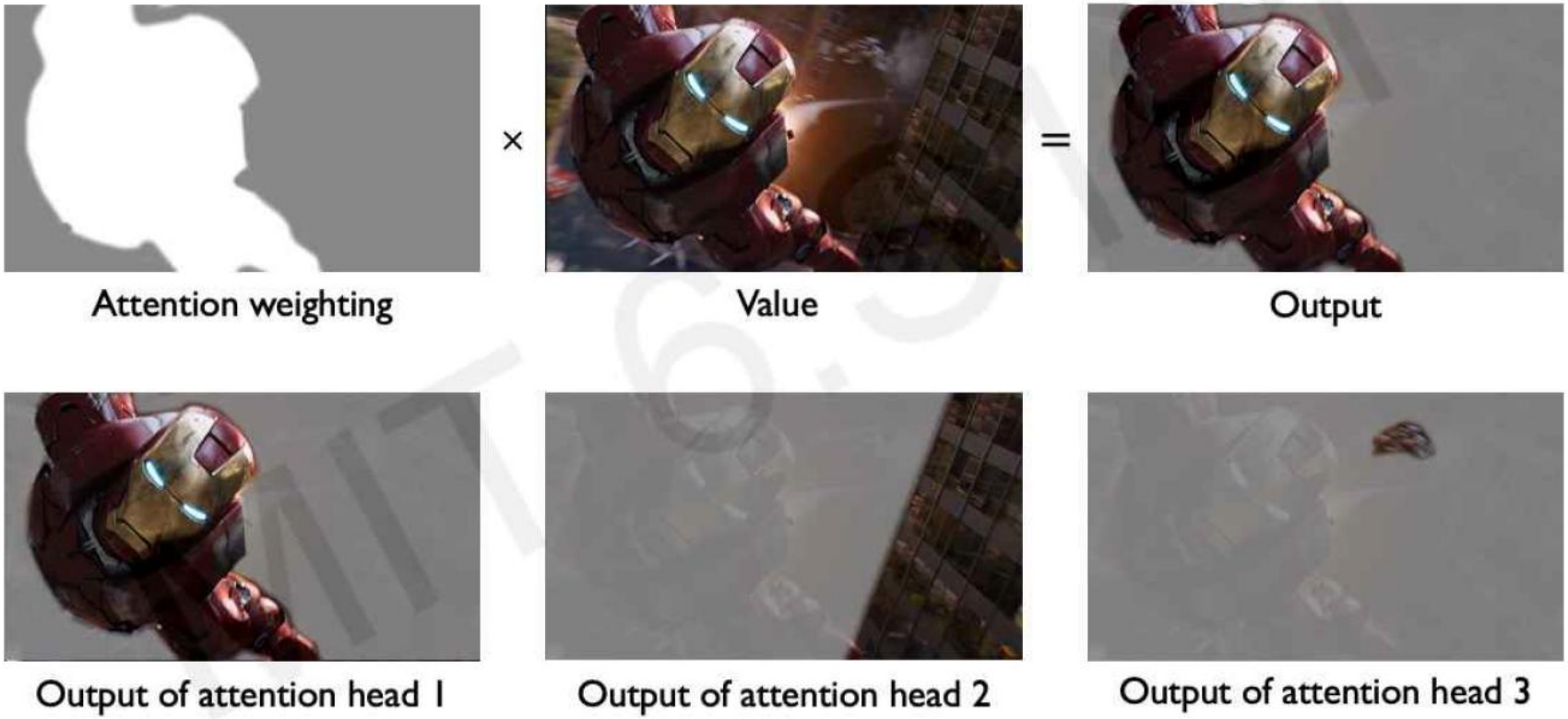
The Limitation:

Single attention collapses competing meanings.

The Solution:

Parallel heads project Q, K, V into smaller, unique sub-spaces.

The Result:

Head 1 tracks pronouns,
Head 2 tracks verbs,
Head 3 tracks sentiment
—simultaneously.

Applying Multiple Self-Attention Heads